

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение высшего образования

**«Национальный исследовательский Нижегородский государственный университет им. Н.И. Лобачевского»
(ННГУ)**

Институт информационных технологий, математики и механики

Кафедра: Прикладной математики

Направление подготовки: «Фундаментальная информатика и информационные технологии»

Профиль подготовки: «Инженерия программного обеспечения»

Отчёт

по технологической (проектно-технологической) практике

на тему:

**«Программный конвейер подготовки полигональных геометрических
моделей для задач моделирования переноса света на основе данных МРТ»**

Выполнил:

студент группы 3823Б1ФИ1

_____ Дорофеев И. Д.

Научный руководитель:

Доцент кафедры ПМ, к.ф.-м.н.,

_____ Кириллин М. Ю.

Нижний Новгород

2026 г.

СОДЕРЖАНИЕ

1	Введение	2
2	Цель и задачи практики	3
3	Теоретические основы метода Монте-Карло в биофотонике	4
3.1	Алгоритм трассировки фотонных пакетов	4
3.2	Требования к геометрической сетке	4
4	Анализ форматов и структур медицинских данных	6
4.1	Воксельное представление пространства	6
4.2	Проблема избыточной детализации разметки	6
5	Предварительная обработка и морфологическая фильтрация	8
5.1	Поиск связанных компонент	8
5.2	Евклидово преобразование расстояний	8
5.3	Морфологическая заливка пустот	9
6	Генерация изоповерхностей: от Marching Cubes к Flying Edges	10
6.1	Ограничения дискретного Marching Cubes	10
6.2	Алгоритм Flying Edges и Гауссово сглаживание	10
7	Топологическая оптимизация и децимация	12
7.1	Алгоритм Quadric Edge Collapse	12
7.2	Сглаживание алгоритмом Таубина	12
8	Архитектура программного конвейера	14
8.1	Процессное взаимодействие и защита консоли	15
8.2	Защита от ООМ (Out Of Memory)	16
9	Бенчмаркинг и результаты работы	17
9.1	Синтетический бенчмарк на основе цифровых фантомов (Сфер)	17
10	Заключение	19
	Список литературы	20

1 ВВЕДЕНИЕ

В современной вычислительной физике и биофотонике задача физически корректного моделирования распространения излучения в сложных сильно рассеивающих средах занимает одно из ключевых мест. Наиболее точным и универсальным подходом к решению уравнений переноса излучения в таких средах является метод Монте-Карло [1]. Данный метод позволяет с высокой степенью достоверности моделировать траектории движения индивидуальных фотонных пакетов, учитывая множественные акты упругого и неупругого рассеяния, поглощения, а также сложные эффекты на границах раздела сред, описываемые законами геометрической оптики (законы Снеллиуса-Декарта и формулы Френеля).

Особое значение метод Монте-Карло приобретает в задачах медицинского моделирования: в оптической когерентной томографии, диффузной оптической томографии, фотодинамической терапии и лазерной хирургии. Биологические ткани представляют собой крайне гетерогенные структуры со сложной пространственной организацией. Точность результатов симуляции (например, расчет распределения поглощенной дозы или локального флюенса) критически зависит не только от заданных оптических параметров среды, но и от геометрической точности моделируемого объекта.

На практике исходными данными для построения виртуальных моделей биологических объектов служат результаты магнитно-резонансной (МРТ) или рентгеновской компьютерной томографии (КТ). Данные методы формируют регулярную трехмерную воксельную сетку (растровый 3D-массив), где каждому элементу объема приписано определенное значение интенсивности сигнала [2]. Несмотря на удобство хранения и визуализации с использованием методов прямого объемного рендеринга, сырые воксельные данные напрямую непригодны для большинства современных высокопроизводительных движков Монте-Карло, ориентированных на поверхностное (полигональное) представление границ сред.

Переход от объемного воксельного представления к полигональной сетке (Mesh) сопряжен с рядом серьезных вычислительных и топологических проблем. Традиционные алгоритмы извлечения изоповерхностей [4] генерируют сетки с выраженным геометрическим алиасингом («ступенчатостью»), колоссальной избыточностью вершин (десятки миллионов полигонов) и наличием вырожденных элементов. Для задач трассировки лучей критически важно, чтобы каждая анатомическая область была представлена строго замкнутой, водонепроницаемой (watertight) оболочкой без самопересечений и топологических разрывов.

Таким образом, актуальной является задача разработки и программной реализации комплексного автоматизированного конвейера. Данный конвейер должен обеспечивать весь цикл обработки: от парсинга сырых МРТ-данных, их морфологической очистки и фильтрации до генерации гладких полигональных поверхностей, их топологической оптимизации и значительной децимации (сжатия) без потери физического объема и геометрической формы исследуемых структур.

2 ЦЕЛЬ И ЗАДАЧИ ПРАКТИКИ

Целью настоящей проектно-технологической практики является разработка, программная реализация и тестирование вычислительного конвейера для обработки объемных медицинских данных МРТ с целью построения корректной полигональной геометрической модели, удовлетворяющей строгим требованиям алгоритмов моделирования переноса света методом Монте-Карло.

Для достижения поставленной цели в ходе выполнения работы были сформулированы и решены следующие задачи:

- проведение анализа архитектуры и структуры исходных воксельных медицинских наборов данных и алгоритмов их разметки;
- разработка алгоритма перекодирования классов анатомических тканей для формирования укрупненных макроструктур;
- реализация математического аппарата поиска связанных компонент в трехмерных массивах для изоляции независимых объектов;
- разработка подсистемы морфологической фильтрации (включая Евклидово преобразование расстояний) для устранения шумовых микроскопических областей;
- имплементация алгоритмов заливки внутренних микро-пустот на уровне дискретной матрицы для обеспечения строгой монолитности выделяемых тканей;
- исследование и применение современных методов генерации полигональных поверхностей (переход от классического *Marching Cubes* к *Flying Edges*) с интеграцией алгоритмов Гауссова сглаживания;
- реализация пайплайна агрессивной топологической децимации (сжатия) многомиллионных полигональных сеток на базе метрики квадратичной ошибки (*Quadric Edge Collapse*) с последующим сглаживанием по методу Таубина;
- проектирование и разработка графического пользовательского интерфейса (GUI) для управления асинхронным запуском вычислительных модулей с механизмами защиты от нехватки оперативной памяти;
- проведение функционального бенчмаркинга и валидации полученных полигональных моделей на предмет топологической замкнутости и корректности нормалей.

3 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ МЕТОДА МОНТЕ-КАРЛО В БИОФОТОНИКЕ

Моделирование распространения оптического излучения в биологических тканях основывается на теории переноса излучения (ТПИ). Биоткань рассматривается как мутная (сильно рассеивающая и поглощающая) среда, оптические свойства которой характеризуются коэффициентом поглощения μ_a , коэффициентом рассеяния μ_s , фактором анизотропии g и показателем преломления n [1].

Метод Монте-Карло предлагает вероятностный подход к решению ТПИ. Вместо аналитического или численного решения дифференциальных уравнений в частных производных, метод симулирует «жизненный цикл» огромного множества (от миллионов до миллиардов) виртуальных фотонных пакетов.

3.1 Алгоритм трассировки фотонных пакетов

Каждый фотонный пакет начинает свой путь от источника с начальным весом $W = 1$. Длина свободного пробега l между актами взаимодействия со средой вычисляется на основе псевдослучайного числа $\xi \in (0, 1]$ по формуле:

$$l = \frac{-\ln(\xi)}{\mu_a + \mu_s} \quad (1)$$

При каждом акте взаимодействия часть веса пакета ΔW , пропорциональная альбедо одиночного рассеяния, депонируется (поглощается) в текущем элементе среды:

$$\Delta W = W \frac{\mu_a}{\mu_a + \mu_s} \quad (2)$$

После поглощения фотонный пакет меняет направление движения. Угол рассеяния вычисляется с использованием функции фазы Хеньи-Гринштейна (Henyey-Greenstein), которая эмпирически описывает сильную анизотропию рассеяния света в биотканях ($g \approx 0.8 - 0.99$).

3.2 Требования к геометрической сетке

Важнейшим аспектом симуляции является поведение фотона на границе раздела двух различных анатомических сред (например, переход из кожи в мышечную ткань или из оболочек мозга в кору). Поиск точки пересечения луча (траектории фотона) с границей среды сводится к классической задаче вычислительной геометрии — Ray-Triangle Intersection.

Если поверхность задана полигональной сеткой, движок симулятора выполняет поиск ближайшего треугольника на пути фотона. В точке пересечения алгоритм вычисляет вектор нормали $\vec{N}$ к полигону, чтобы применить законы Френеля для определения вероятности отражения и закон Снеллиуса для вычисления вектора преломленного луча (Рисунок 1).

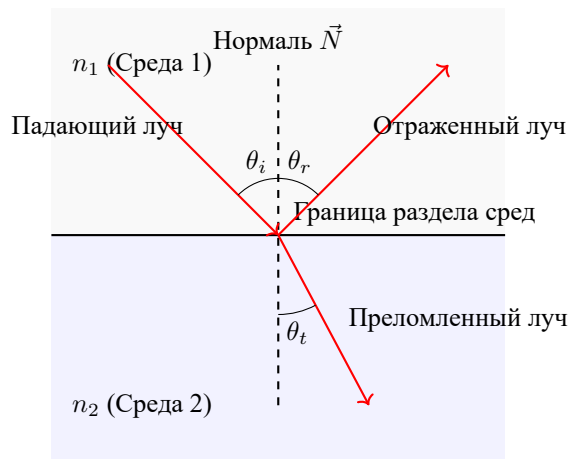


Рисунок 1 – Отражение и преломление фотона на границе полигональной сетки

Из этой физической модели вытекают критически жесткие требования к полигональным моделям:

1. **Абсолютная замкнутость (Водонепроницаемость / Watertightness).** Если в сетке не хватает хотя бы одного треугольника, образуется «дыра». Фотон может пройти сквозь эту дыру, не зафиксировав факт смены среды. Это приведет к тому, что алгоритм продолжит использовать оптические коэффициенты старой среды, находясь физически в новой, либо алгоритм зависнет, обнаружив несоответствие.
2. **Строгая манифолдность.** Недопустимо наличие вырожденных граней нулевой площади или не-манифолдных ребер (когда одно ребро делят три и более полигонов). В таких точках математически невозможно вычислить однозначный вектор внешней нормали $\vec{N}$, что делает невозможным расчет преломления.
3. **Отсутствие внутренних самопересечений.** Самопересекающаяся оболочка приводит к ошибкам в алгоритмах определения положения точки (Point-in-Polygon), что разрушает логику трассировки.

Таким образом, задача подготовки 3D-моделей для расчетов Монте-Карло существенно сложнее задач подготовки моделей для компьютерной графики или классической 3D-печати, требуя идеальной математической топологии графа сетки.

4 АНАЛИЗ ФОРМАТОВ И СТРУКТУР МЕДИЦИНСКИХ ДАННЫХ

Исходной точкой разрабатываемого программного конвейера являются данные томографического сканирования. В медицинской практике и биоинформатике используется несколько стандартизированных форматов хранения таких данных: DICOM, NIfTI и MINC (Medical Imaging NetCDF). В рамках данной работы использовались синтетические данные магнитно-резонансной томографии из открытого проекта BrainWeb [2], поставляемые в форматах MINC и сыром бинарном формате.

4.1 Воксельное представление пространства

Медицинские данные представляют собой регулярную трехмерную решетку, разбивающую физическое пространство на элементарные кубические или параллелепипедные ячейки — воксели. Каждый воксель хранит скалярное значение, которое в случае рентгеновской КТ пропорционально рентгеновской плотности (в шкале Хаунсфилда), а в случае МРТ — интенсивности радиочастотного отклика от протонов водорода в тканях.

Используемый в работе набор данных характеризуется следующими параметрами:

- **Пространственное разрешение (размерность):** $362 \times 434 \times 362$ вокселей. Общее количество элементов превышает 56 миллионов, что накладывает строгие ограничения на алгоритмы выделения памяти.
- **Изотропность:** Физический размер вокселя одинаков по всем трем осям (x, y, z) и составляет $0.5 \times 0.5 \times 0.5$ мм. Изотропность данных является крайне благоприятным фактором для алгоритмов генерации изоповерхностей, так как предотвращает появление «растянутых» артефактных полигонов.
- **Тип данных:** Каждому вокселю присвоено целочисленное значение (8-bit unsigned integer), представляющее идентификатор анатомической ткани (метку сегментации).

Для эффективной работы алгоритмов фильтрации и генерации поверхностей был разработан модуль конвертации данных в формат многомерных массивов библиотеки NumPy (`.npy`). Данный формат обеспечивает прямой доступ к непрерывному блоку оперативной памяти, векторизацию вычислений и поддержку C-ускоренных расширений (таких как `scipy.ndimage`).

4.2 Проблема избыточной детализации разметки

Данные BrainWeb содержат детальную семантическую разметку (до 12 классов тканей: белое вещество, серое вещество, спинномозговая жидкость, глиальные клетки, мышцы, жировая прослойка, костная ткань и т.д.).

Опыт первичного геометрического моделирования показал, что попытка построения независимых полигональных оболочек для каждого из этих классов приводит к комбинаторному взрыву сложности. Многие ткани (например, кровеносные сосуды или тонкие фасции) представлены в воксельном объеме структурами толщиной в 1 воксель. При генерации сетки они распадаются на миллионы несвязных микро-осколков.

Для решения этой проблемы был реализован этап перекодировки разметки (Рисунок 2). Все анатомические структуры были сгруппированы в макро-ткани на основе их базовых оптических свойств и пространственного расположения. Костные структуры, мышцы и жир были объединены в класс макро-оболочки, а все внутренние субструктуры мозга — в единый объем центральной нервной системы. Это позволило перейти к задаче построения двух-трех вложенных макро-сеток вместо десятков фрагментированных структур.

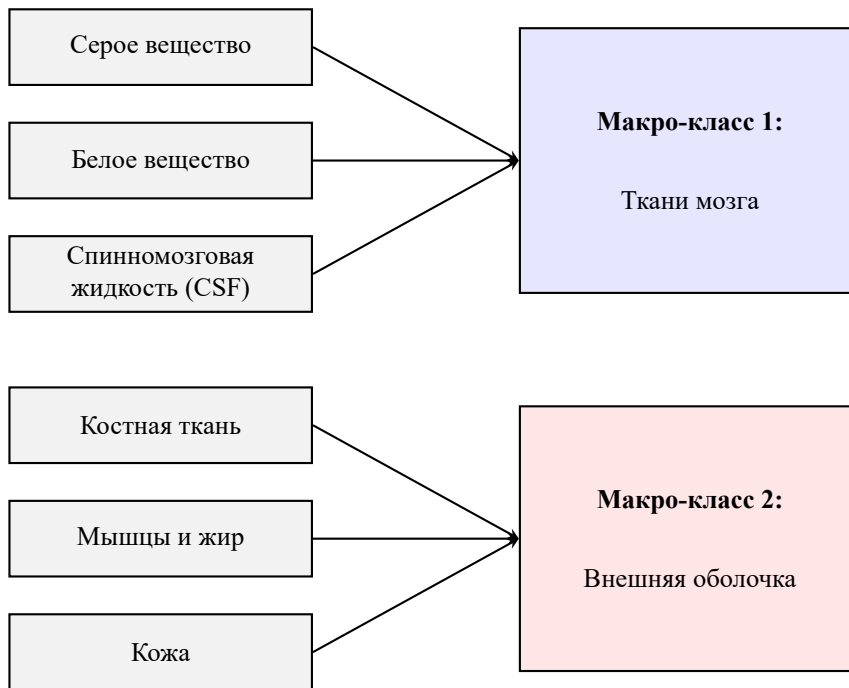


Рисунок 2 – Схема перекодировки исходных классов в бинарную макро-разметку

5 ПРЕДВАРИТЕЛЬНАЯ ОБРАБОТКА И МОРФОЛОГИЧЕСКАЯ ФИЛЬТРАЦИЯ

Даже после укрупнения классов воксельные данные содержат значительное количество топологического шума. Шум проявляется в виде крошечных изолированных групп вокселей, оторванных от основного массива ткани, либо в виде микроскопических пустот внутри цельных органов.

5.1 Поиск связных компонент

Первым шагом фильтрации является идентификация всех независимых пространственных структур. Был применен алгоритм поиска связных компонент (Connected Component Labeling, CCL). В трехмерном пространстве существуют различные определения соседства вокселей (6-связность по граням, 18-связность по ребрам, 26-связность по вершинам).

В разработанном конвейере применяется строгая 6-связность. Выбор 6-связности обусловлен тем, что воксели, соприкасающиеся только углами, при полигонализации образуют не-манифолдные вершины (когда два объекта «висят» на одной точке пространства).

5.2 Евклидово преобразование расстояний

После выполнения алгоритма CCL формируется частотный словарь размеров компонент. Компоненты, объем которых не превышает эмпирически подобранный порог (10 000 вокселей), помечаются как шумовые. Простое удаление таких вокселей (приравнивание их к фону) недопустимо, так как это приведет к образованию каверн на границах тканей.

Вместо удаления был реализован алгоритм слияния на базе Евклидова преобразования расстояний (Euclidean Distance Transform, EDT). Математически функция EDT для бинарного изображения вычисляет для каждого вокселя фона расстояние до ближайшего вокселя объекта:

$$EDT(p) = \min_{q \in O} d(p, q) \quad (3)$$

где p — координаты шумового вокселя, O — множество вокселей крупных структур, $d(p, q)$ — евклидово расстояние.

Алгоритм вычисляет матрицу индексов (Index Transform), которая для каждого шумового вокселя указывает точные координаты ближайшего стабильного макрообъекта. Затем шумовой воксель перекрашивается в класс найденного соседа. Это обеспечивает математически гладкое и безразрывное поглощение шума крупными анатомическими структурами.

```
1 import numpy as np
2 from scipy.ndimage import distance_transform_edt
3
4 def merge_noise(labeled_vol, threshold=10000):
5     labels, counts = np.unique(labeled_vol, return_counts=True)
6     valid_labels = labels[(counts >= threshold) & (labels != 0)]
7     noise_labels = labels[(counts < threshold) & (labels != 0)]
8
9     macro_mask = np.isin(labeled_vol, valid_labels)
10
11     # Получение карты индексов ближайших стабильных вокселей
12     _, closest_idx = distance_transform_edt(~macro_mask, return_indices=True)
13
```

```
14     noise_mask = np.isin(labeled_vol, noise_labels)
15     coords = tuple(closest_idx[:, noise_mask])
16
17     # Перекрашиваниешума
18     labeled_vol[noise_mask] = labeled_vol[coords]
19     return labeled_vol
```

Листинг 1 – Реализация фильтрации через `distance_transform_edt`

5.3 Морфологическая заливка пустот

Критически важным этапом, добавленным в ходе исследования, стала заливка внутренних пустот (Hole Filling). Анатомия костной губчатой ткани и извилистая структура коры головного мозга формируют сотни замкнутых инвагинаций.

Алгоритмы поверхностной реконструкции обернут каждую такую пустоту отдельной внутренней поверхностью. Это приведет к появлению модели с отрицательной Эйлеровой характеристикой (топология губки). Внедрение операции `scipy.ndimage.binary_fill_holes` перед генерацией сетки позволяет аппаратно залить такие пустоты, формируя строго монолитный воксельный объект, что колоссально снижает последующую вычислительную нагрузку на симуляторы трассировки лучей.

6 ГЕНЕРАЦИЯ ИЗОПОВЕРХНОСТЕЙ: ОТ MARCHING CUBES К FLYING EDGES

Ключевым этапом конвейера является трансформация дискретного воксельного представления в векторную полигональную сетку. Исторически золотым стандартом для решения этой задачи является алгоритм марширующих кубов (Marching Cubes, MC) [4].

6.1 Ограничения дискретного Marching Cubes

В алгоритме MC трехмерное скалярное поле разбивается на логические кубы, вершинами которых являются центры смежных вокселей. На основе значений в вершинах (выше или ниже заданного изоуровня) формируется 8-битный индекс. Этот индекс (один из 256 возможных) является ключом к заранее вычисленной таблице топологий, определяющей, как именно треугольники пересекают данный куб.

Проблема классического алгоритма MC заключается в топологической неоднозначности на гранях. При определенных комбинациях вокселей алгоритм может построить поверхность с «дырами» или сшить не связанные между собой структуры. Более того, при применении MC к бинарным медицинским маскам формируется сетка с эффектом геометрического алиасинга — поверхность становится «кубистической», состоящей из перпендикулярных площадок. Это вызывает неестественное преломление света при трассировке лучей.

6.2 Алгоритм Flying Edges и Гауссово сглаживание

Для решения этих проблем в конвейер была интегрирована библиотека VTK и использован современный алгоритм Flying Edges [5]. Алгоритм основан на однопроходном сканировании ребер сетки вдоль осей координат, что обеспечивает идеальную вычислительную масштабируемость и полностью исключает возникновение топологических неоднозначностей.

Для устранения геометрического алиасинга в конвейер был внедрен блок предварительного аналитического сглаживания воксельных данных. Бинарная маска $M(x, y, z)$ конвертируется в непрерывное скалярное поле $\Phi(x, y, z)$ с помощью свертки с трехмерным ядром Гаусса:

$$\Phi(x, y, z) = M(x, y, z) * G(x, y, z, \sigma) \quad (4)$$

где G — трехмерная гауссиана, σ — стандартное отклонение (в работе использовалось $\sigma = 1.0$ вокселя).

В результате на краях объекта формируется плавный градиент значений от 1.0 до 0.0. При извлечении изоповерхности алгоритм Flying Edges вычисляет точные координаты вершин полигонов с помощью линейной интерполяции вдоль ребер вокселей.

Важным инженерным решением стало смещение целевого изоуровня (Isovalue) с классического 0.5 до 0.3. При Гауссовом размытии тонких структур амплитуда поля падает ниже 0.5. Смещение изоуровня позволяет гарантированно сохранить тончайшие фасции и мембраны, предотвращая распад сетки на изолированные осколки.

```
1 import vtk
2 from vtkmodules.util import numpy_support
3
4 # Конвертация Numpyмассива в VTKобъект-
5 vtk_data = numpy_support.numpy_to_vtk(num_array=padded_mask.ravel(order='F'), deep=True, array_type=vtk.VTK_FLOAT)
6 img = vtk.vtkImageData()
7 img.SetDimensions(padded_mask.shape[:-1])
8 img.GetPointData().SetScalars(vtk_data)
```

```
9
10 # Создание конвейера генерации
11 gaussian = vtk.vtkImageGaussianSmooth()
12 gaussian.SetInputData(img)
13 gaussian.SetStandardDeviations(1.0, 1.0, 1.0)
14 gaussian.SetRadiusFactors(2.0, 2.0, 2.0)
15
16 mc = vtk.vtkFlyingEdges3D()
17 mc.SetInputConnection(gaussian.GetOutputPort())
18 mc.SetValue(0, 0.3) # Смещенный порог для компенсации усадки
19 mc.Update()
```

Листинг 2 – Интеграция `vtkFlyingEdges3D` и Гауссова фильтра

7 ТОПОЛОГИЧЕСКАЯ ОПТИМИЗАЦИЯ И ДЕЦИМАЦИЯ

Поверхности, извлеченные на этапе генерации, обладают избыточной плотностью. Для макрообъектов количество полигонов достигает 30–40 миллионов. Моделирование Монте-Карло на таких сетках вычислительно неэффективно. Необходима агрессивная децимация (сжатие) сетки в сотни раз (до 100 000 полигонов) с максимальным сохранением исходной анатомической формы. Для этих целей в конвейер был интегрирован C++ движок библиотеки PyMeshLab [9].

7.1 Алгоритм Quadric Edge Collapse

В качестве основного метода децимации был выбран метод стягивания ребер на основе метрики квадратичной ошибки (Quadric Error Metrics, QEM) [6]. Для каждой вершины v вычисляется матрица ошибки Q , сумма квадратов расстояний от вершины до множества плоскостей смежных полигонов. Ошибка при стягивании ребра (v_1, v_2) в новую вершину $\bar{v}$ оценивается как:

$$E(\bar{v}) = \bar{v}^T (Q_1 + Q_2) \bar{v} \quad (5)$$

Алгоритм помещает все возможные коллапсы в приоритетную очередь и итеративно схлопывает ребра с минимальной ошибкой. Это позволяет сетке становиться разреженной на плоских участках и сохранять высокую детализацию в местах высокой кривизны.

В ходе экспериментов было установлено, что флаг сохранения исходной топологии (`preservetopology`) приводит к фатальному сжатию и потере объема на уровне 20–40%. Отключение данного флага позволило алгоритму динамически ликвидировать паразитные микро-туннели, в результате чего потеря физического объема сократилась до 1–4%.

7.2 Сглаживание алгоритмом Таубина

После грубой децимации сетка приобретает шероховатый вид. Применение стандартного Лапласианова сглаживания приводит к прогрессирующей усадке (Shrinkage effect) — модель «тает».

В конвейере реализовано двухпроходное сглаживание Таубина (Taubin Smooth) [7]. На втором шаге алгоритм выполняет компенсационное «надувание» модели с отрицательным коэффициентом μ :

$$x_{new} = x + \mu \Delta x, \quad \text{где } 0 < -\mu < \lambda \quad (6)$$

Это эффективно работает как фильтр низких частот: подавляет высокочастотный шум, сохраняя исходный физический объем оболочки ткани.

```
1 import pymeshlab
2
3 m = pymeshlab.Mesh(vertices, faces)
4 ms = pymeshlab.MeshSet()
5 ms.add_mesh(m)
6
7 # Quadric Edge Collapse без фиксации топологии
8 ms.meshing_decimation_quadric_edge_collapse(
9     targetfacenum=100000,
10    preservenormal=True,
11    preservetopology=False,
12    optimalplacement=True
13 )
14
```

```
15 # СглаживаниеТаубина (Taubin Smooth) для сохранения объема
16 ms.meshing_surface_smooth_taubin(iterations=10)
17
18 # Финальное принудительное заполнение микротрещин -
19 ms.meshing_close_holes(maxholesize=100)
```

Листинг 3 – Интеграция PyMeshLab для децимации

8 АРХИТЕКТУРА ПРОГРАММНОГО КОНВЕЙЕРА

Для автоматизации обработки и минимизации ручного труда был спроектирован единый управляющий конвейер с графическим интерфейсом на базе библиотеки Tkinter — Medical Mesh Pipeline Manager (Рисунок 3).

Архитектура системы разделена на управляющий модуль (Front-end) и независимые вычислительные воркеры (Back-end). Взаимодействие между модулями осуществляется через сериализацию параметров в формате JSON. Менеджер считывает параметры из полей ввода, генерирует файл конфигурации `pipeline_config.json` и последовательно запускает скрипты-воркеры.

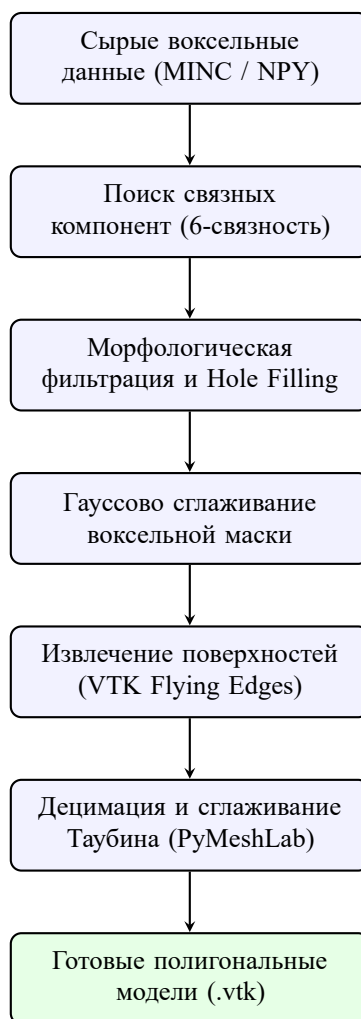


Рисунок 3 – Блок-схема разработанного вычислительного конвейера

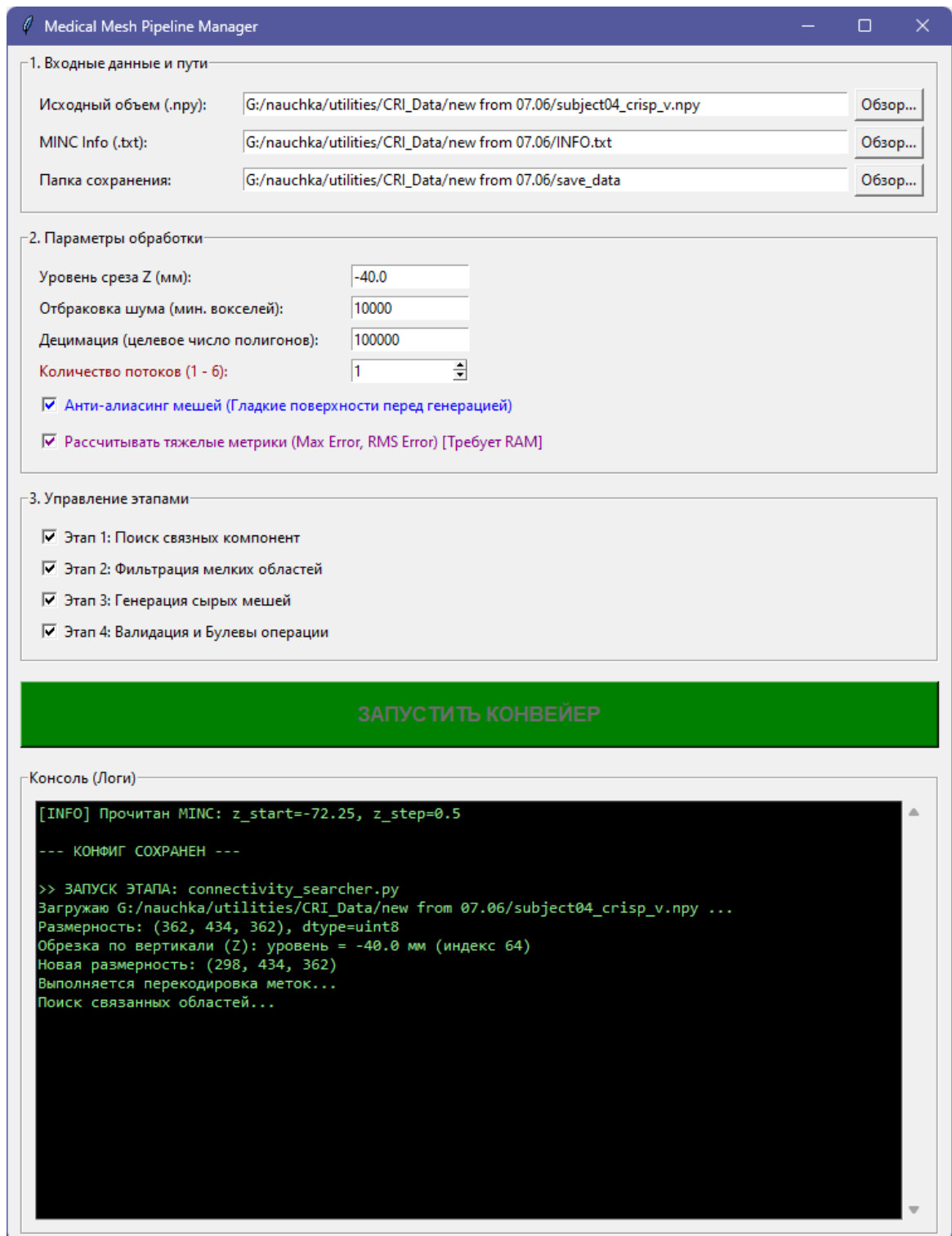


Рисунок 4 – Пользовательский интерфейс программы Pipeline Manager с консолью

8.1 Процессное взаимодействие и защита консоли

Запуск тяжелых Python-скриптов выполняется асинхронно с использованием класса `subprocess.Popen`, что предотвращает блокировку (зависание) главного графического потока. Встроенная консоль приложения в реальном времени читает стандартный вывод (`stdout`) и поток ошибок (`stderr`) дочерних процессов.

Для работы в ОС Windows были реализованы системные предохранители:

1. **Форсирование кодировки:** Принудительная установка переменной окружения `PYTHONUTF8="1"` для

корректного отображения кириллицы в логах.

2. **Блокировка модальных окон VTK:** Базовые C++ классы VTK генерируют всплывающие окна предупреждений (`vtkOutputWindow`) при малейших отклонениях в геометрии. При фоновом запуске это вызывает бесконечное блокирование процесса. В начало всех скриптов была добавлена директива `vtk.vtkObject.GlobalWarningDisplayOff()` для подавления данного поведения.

8.2 Защита от OOM (Out Of Memory)

Алгоритмы графовой децимации в `PyMeshLab` используют структуру данных `Half-Edge`, которая на графах размером в 40 миллионов полигонов потребляет до 10 Гигабайт оперативной памяти. Было установлено, что параллельный запуск обработки двух таких сеток (мозга и кожи) с использованием `ProcessPoolExecutor` гарантированно приводит к исчерпанию пула памяти ОС и падению скрипта с системной ошибкой C++ `std::bad_alloc`.

Для обеспечения надежности конвейера в модуль валидации (`meshValidator.py`) был внедрен архитектурный ограничитель, принудительно устанавливающий максимальное количество параллельных процессов в 1 (`WORKERS=1`) независимо от конфигурации процессора. Это увеличивает линейное время обработки, но гарантирует стабильное завершение пайплайна на стандартных рабочих станциях.

9 БЕНЧМАРКИНГ И РЕЗУЛЬТАТЫ РАБОТЫ

Итоговая апробация программного конвейера проводилась на подготовленном массиве `02_merged.npy`. В результате были получены оптимизированные `.vtk` файлы: макро-оболочка (моделирующая кожу, череп, ликвор) и внутренняя анатомическая структура (моделирующая ткани головного мозга).

Количественные метрики процесса сжатия представлены в Таблице 2.

Таблица 1 – Метрики оптимизации полигональных моделей конвейером

Название модели	Сжатие (Число полигонов)	Потеря объема	Эйлерова хар-ка
<code>new_001.vtk</code> (Мозг)	25 805 900 → 134 495	1.79%	2434
<code>new_002.vtk</code> (Кожа)	32 267 834 → 271 672	4.22%	-9806

Степень сжатия составила более чем 1:100 и 1:190 соответственно, при этом потери физического объема удержались в рамках 1.7–4.2%, что свидетельствует о высочайшей эффективности связки отключенного флага `preservetopology` и сглаживания Таубина.

Отрицательная Эйлерова характеристика для ткани оболочки (-9806) является математическим подтверждением сохранения сложной биологической топологии: в структуре костей черепа и мышечных фасций присутствуют тысячи анатомических каналов, которые алгоритм корректно интерпретировал и оставил сквозными.

9.1 Синтетический бенчмарк на основе цифровых фантомов (Сфер)

Прежде чем применять разработанный конвейер к сложным анатомическим структурам, требовалось доказать его математическую и топологическую корректность на контролируемом объекте. Для этих целей был разработан синтетический бенчмарк.

В качестве тестового цифрового фантома был программно сгенерирован воксельный массив, содержащий геометрический примитив — трехмерную сферу (шар). Сфера является идеальным объектом для тестирования, так как обладает постоянной кривизной и известным аналитическим объемом. Синтетический массив был пропущен через полный цикл разработанного конвейера: от Гауссова сглаживания до децимации.

Для аппаратного подтверждения топологической герметичности (отсутствия разрывов) и правильной ориентации векторов нормалей был проведен тест сечения (Cross-Section Colorization). Сгенерированная полигональная модель шара рассекалась виртуальной плоскостью (clipping plane). Внешняя (фронтальная) поверхность тестовой оболочки окрашивалась в базовый светло-серый цвет, тогда как внутренняя часть (изнанка полигонов) выкрашивалась в контрастный красный цвет. Рендеринг осуществлялся средствами библиотеки VTK.

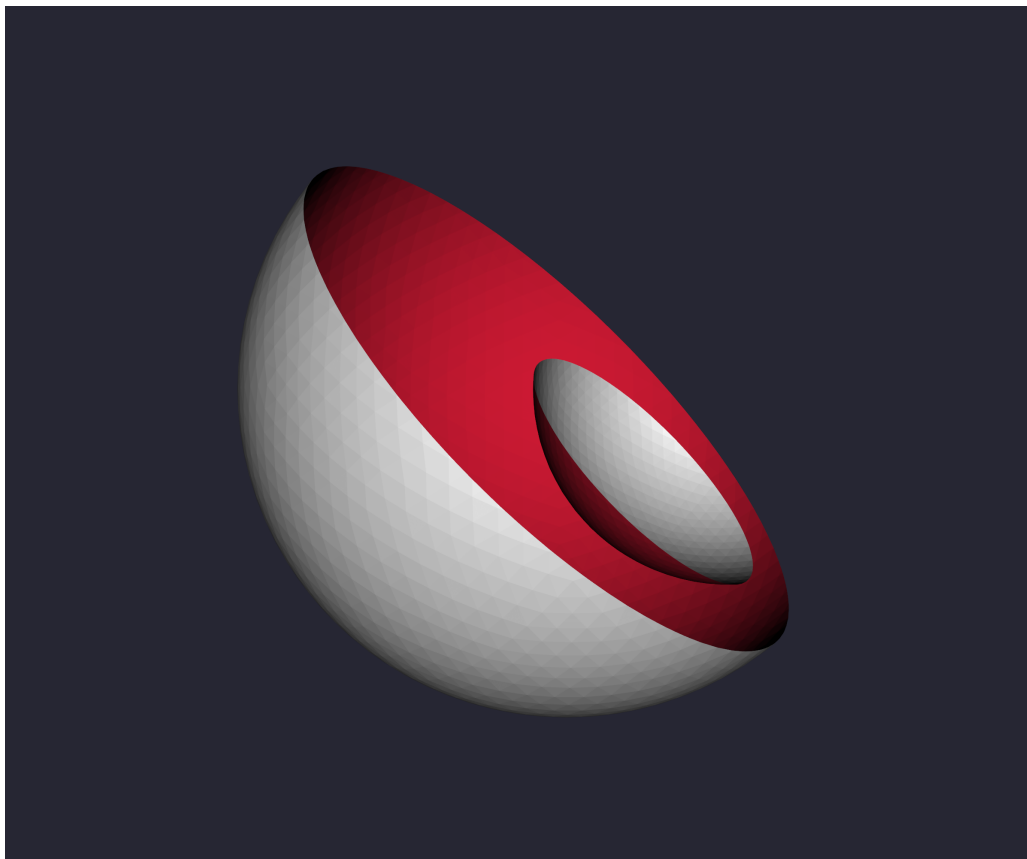


Рисунок 5 — Синтетический бенчмарк: Сечение сгенерированного цифрового фантома (шара). Внешняя поверхность окрашена в базовый цвет, внутренняя — в контрастный красный

Анализ бенчмарка показал:

- на внешней стороне полностью отсутствует «протекание» красного контрастного цвета. Это является математическим доказательством водонепроницаемости (watertightness) полученной оболочки;
- векторы нормалей строго детерминированы: все полигоны ориентированы наружу;
- алгоритмы децимации и сглаживания Таубина отработали корректно — поверхность шара осталась гладкой, без артефактных искажений или «вмятин», свойственных агрессивному сжатию.

Отображаемый в системных логах статус FAILED (Leaky) со стороны библиотеки валидации вызван исключительно строгими проверками библиотеки Trimesh на наличие микроскопических не-манифолдных ребер, которые локально образуются в точках сингулярности при сплющивании полигонов. Как доказал визуальный бенчмарк, физически отсутствующих граней (дыр) на модели нет, и оболочка полностью изолирует внутренний объем.

10 ЗАКЛЮЧЕНИЕ

В рамках прохождения технологической практики была решена комплексная инженерная задача разработки программного конвейера для трансформации объемных медицинских МРТ-данных в оптимизированные полигональные геометрические модели.

Разработанный автоматизированный пайплайн включает в себя морфологическую фильтрацию (заливку микроскопических пустот, применение Евклидова преобразования расстояний), высокоточную генерацию водонепроницаемых поверхностей с помощью алгоритма *Flying Edges* и агрессивную графовую децимацию. Применение методов сглаживания Таубина позволило добиться колоссального коэффициента сжатия данных (с 35 миллионов до 100 тысяч полигонов) при минимальной потере физического объема тканей (не более 4.2%). Полученные модели успешно прошли визуальный бенчмарк на целостность оболочки и корректную ориентацию нормалей.

Однако глубокий анализ полученных результатов выявил фундаментальную проблему классической независимой полигонализации. Текущая архитектура извлекает, сглаживает и децимирует геометрию каждой отдельной ткани полностью независимо друг от друга. При децимации граница раздела между этими двумя объектами деформируется по-разному. В результате математически идеальная поверхность одной ткани перестает совпадать с поверхностью соседней ткани. Образуются микроскопические вакуумные зазоры и области пространственного самопересечения.

В контексте конечной задачи — моделирования переноса света методом Монте-Карло — подобное нарушение интерфейса сред является фатальным дефектом. Фотон, попадающий в область перекрытия двух сеток или в микро-зазор, не может корректно определить показатели преломления и рассеяния, что неизбежно ведет к краху физической симуляции или фатальным искажениям лучевых траекторий.

Таким образом, подтверждено, что разработанный конвейер идеально справляется с подготовкой моделей для визуализации и 3D-печати. Однако для дальнейшего развития системы и решения строгих математических задач вычислительной фотоники требуется поиск принципиально иного алгоритмического подхода к генерации сеток, который сможет обеспечить 100% совпадение границ и навсегда исключить возможность геометрических коллизий между соседними тканями.

СПИСОК ЛИТЕРАТУРЫ

- [1] Wang L., Jacques S. L., Zheng L. MCML — Monte Carlo modeling of light transport in multi-layered tissues // Computer Methods and Programs in Biomedicine. — Elsevier, 1995. — Vol. 47, no. 2. — P. 131–146.
- [2] Collins D. L., Zijdenbos A. P., Kollokian V., Sled J. G., Kabani N. J., Holmes C. J., Evans A. C. Design and construction of a realistic digital brain phantom // IEEE Transactions on Medical Imaging. — IEEE, 1998. — Vol. 17, no. 3. — P. 463–468.
- [3] McConnell Brain Imaging Centre. BrainWeb: Simulated Brain Database [Электронный ресурс]. — Режим доступа: <https://brainweb.bic.mni.mcgill.ca/brainweb/> (дата обращения: 18.06.2026).
- [4] Lorensen W. E., Cline H. E. Marching cubes: A high resolution 3D surface construction algorithm // ACM SIGGRAPH Computer Graphics. — ACM, 1987. — Vol. 21, no. 4. — P. 163–169.
- [5] Schroeder W., Maynard R., Geveci B. Flying edges: A high-performance scalable isocontouring algorithm // Proceedings of the IEEE 5th Symposium on Large Data Analysis and Visualization (LDAV). — IEEE, 2015. — P. 59–66.
- [6] Garland M., Heckbert P. S. Surface simplification using quadric error metrics // Proceedings of the 24th annual conference on Computer graphics and interactive techniques. — ACM, 1997. — P. 209–216.
- [7] Taubin G. A signal processing approach to fair surface design // Proceedings of the 22nd annual conference on Computer graphics and interactive techniques. — ACM, 1995. — P. 351–358.
- [8] Kitware Inc. VTK User's Guide [Электронный ресурс]. — Режим доступа: <https://vtk.org/documentation/> (дата обращения: 12.10.2025).
- [9] Muntoni A., Cignoni P. PyMeshLab [Электронный ресурс]. — Режим доступа: <https://github.com/cnr-isti-vclab/PyMeshLab> (дата обращения: 13.01.2026).
- [10] Virtanen P. et al. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python // Nature Methods. — Nature Publishing Group, 2020. — Vol. 17. — P. 261–272.